

Sequence Matching Technique from DNA Expression for Disease Detection

*A project implementing hardware-accelerated DNA sequence matching using
Verilog HDL, simulated in Vivado. It performs sequence alignment
using three progressively advanced approaches
from a simple inline counter to a full
Smith-Waterman local alignment engine.*

**By
Mannvit Kumar**

ABSTRACT

This report is based on the topic “**Sequence Matching Technique from DNA Expression for Disease Detection**”. DNA sequence matching is a crucial technique enabling the identification of similarities and differences between a query DNA sequence and a reference DNA in a database. This process can be majorly used for disease detection, disease diagnosis, evolutionary study and genetic research. Our main goal is to use Smith-Waterman algorithms for global and local sequence alignments.

This report presents an approach to disease detection through sequence matching techniques applied to DNA expressions, utilizing **Vivado** and **Verilog** for implementation. The increasing need for efficient and accurate genetic disease detection has motivated us to achieve our goal.

The methodology involves designing and implementing sequence matching algorithms in Verilog, simulating and verifying them in Vivado, and deploying them on FPGA hardware to assess their real-time performance. Key innovations include the optimization of sequence alignment algorithms for hardware acceleration and the integration of parallel processing capabilities to enhance throughput.

1. INTRODUCTION

1.1 DNA sequence alignment

It is a critical task in bioinformatics, playing a vital role in genetic research, evolutionary studies, and disease detection. Efficiently matching DNA sequences enables researchers to identify genetic mutations, determine evolutionary relationships, and analyze biological data at a large scale. This process involves comparing a query DNA sequence—such as one obtained from a patient—against a reference database containing sequences associated with specific diseases, genetic traits, or evolutionary information.

The primary objective of DNA sequence matching is to detect similarities and differences between the query and reference sequences. These insights help in diagnosing genetic disorders, understanding functional regions of DNA, and guiding personalized medical treatments. For example, identifying a match between a patient's DNA sequence and a known disease-associated mutation can facilitate early diagnosis and targeted therapeutic strategies.

1.2 Previous Work Results and Insights

1.2.1. Simulation in Vivado

You used **Vivado** to simulate the DNA sequence matching logic. This involved designing the **Verilog code**, compiling it, and running testbenches to verify the correctness of the implemented logic. The simulation ensured that the **sequence detector** functioned as expected before proceeding to hardware implementation. It helped in identifying and resolving potential issues in the design.

1.2.2. 4-bit Match Logic for BCD Sequence Comparison

The goal is to design and analyze a 4-bit comparator system to efficiently compare an array of 100 different 4-bit Binary Coded Decimal (BCD) numbers against a predefined reference value. It will identify matches between each element in the array and the reference value. After simulating the modules, the individual matches can be identified.

1.2.3. 7-bit ASCII Comparator for Sequence Matching of Gene Expressions

The goal is to design and implement a 7-bit ASCII comparator specifically tailored for sequence matching in gene expression analysis. A comparator is to be created which is capable of comparing 7-bit ASCII encoded characters representing nucleotides: A (65), T (84), C (67), and G (71) to an array of 100 7-bit numbers. After simulating the modules, the nucleotides can be identified.

1.2.4. 7-Bit ASCII character ATGC sequence matching of DNA expression

The goal is to develop and validate a sequence detection system for identifying specific DNA expression patterns within a given sequence of genetic data. This system will utilize a Sequence Detector to process an input array consisting of 100 characters, each represented as a 7-bit ASCII value corresponding to the nucleotides adenine (A), thymine (T), cytosine (C), and guanine (G). The primary goal of this project is to accurately detect the position of the target DNA sequence 'ATTCCTTTTAAAAGGGA' within the larger 100-character sequence. After simulating the modules, the position of the required DNA sequence can be identified.

1.2.5. Pattern Matching Methodology

To efficiently detect DNA sequences, you explored various pattern matching techniques such as shift registers and counters. By using a sliding window approach, the system checked for pattern matches while processing large DNA datasets. The method allowed for quick identification of target sequences without requiring excessive computational power.

1.2.6. Performance Analysis

You evaluated the performance of the system based on accuracy, speed, and computational efficiency. The assessment included testing the sequence detector's ability to correctly identify target patterns, analyzing its processing time, and ensuring it met real-time execution requirements. The results demonstrated that FPGA-based sequence matching significantly improves efficiency compared to software-based approaches.

1.3 Sequence Alignment Algorithms

DNA sequence matching relies on specialized algorithms to compare sequences efficiently. Two primary sequence alignment approaches are:

(i) Global Alignment (Needleman-Wunsch Algorithm):

This method aligns sequences from end to end, making it useful for comparing sequences of similar length. It systematically explores all possible alignments and determines the best overall match using dynamic programming.

(ii) Local Alignment (Smith-Waterman Algorithm):

Unlike global alignment, this method focuses on identifying the most relevant matching regions within larger sequences. Smith-Waterman is particularly valuable for detecting disease-related genetic markers, mutations, insertions, and deletions. It optimizes for the highest-scoring local alignments, making it a robust approach for analyzing complex biological sequences. Due to its

precision and flexibility, it is widely used in genomics for tasks such as identifying mutations, detecting similarities in protein-coding regions, and aligning short-read DNA sequences in high-throughput sequencing applications.

1.4 Significance of DNA Sequence Matching

DNA sequence matching plays a pivotal role in modern genomics and personalized medicine. Its applications include:

Disease Mutation Detection: By comparing a patient's DNA sequence with known pathogenic variants, Smith-Waterman helps in accurately identifying disease-associated mutations.

Genetic Relationship Analysis: Sequence matching aids in studying genetic relationships and evolutionary links between species.

Personalized Medical Treatments: DNA-based diagnostics help tailor medical treatments to an individual's genetic profile, improving treatment efficacy and minimizing side effects.

By leveraging FPGA technology, the Smith-Waterman algorithm can be significantly accelerated, enabling real-time analysis of large-scale genomic data. The following sections will delve into the methodology, Verilog-based FPGA design, performance analysis, and applications of these hardware-based implementations in bioinformatics.

1.5 Dataset Description

1.5.1 Dataset Overview

For this project, we used the DNA Sequence Dataset available on Kaggle. This dataset consists of a collection of DNA sequences derived from various organisms and genetic studies. It is widely used in bioinformatics research for sequence alignment, mutation detection, and disease prediction.

1.5.2 Data Format and Structure

File Format: CSV (Comma-Separated Values)

Columns: The dataset contains DNA sequences labeled with different categories such as genetic expressions or specific gene sequences.

Sequence Length: The DNA sequences vary in length, making it a suitable dataset for both short-read and long-read sequence alignment.

1.5.3 Data Preprocessing for FPGA Implementation

Since our DNA sequence matching system processes 7-bit ASCII representations, we performed the following preprocessing steps:

- Extract DNA Sequences – Only sequences consisting of nucleotides A, T, C, G were selected.
- Convert to ASCII Encoding – Each nucleotide was replaced with its corresponding 7-bit ASCII value (A = 65, T = 84, C = 67, G = 71).
- Store in External Files – The converted sequences were saved in dna_char.txt (reference sequences) and target.txt (query sequences) for FPGA processing

2. LITERATURE SURVEY

2.1 "A Novel Pattern Matching Algorithm in Genome Sequence Analysis" [1]

It focuses on the development of a new pattern matching algorithm, designed to improve the detection of DNA sequences associated with diseases. Pattern matching is crucial in bioinformatics for identifying specific sequences within large DNA databases, which can be indicative of certain diseases.

Analysis:

•Methodology:

The study evaluates the performance of four existing pattern matching algorithms:

1. Brute Force Algorithm:

- No pre-processing phase.
- Requires constant extra space.
- Always shifts the window by exactly one position to the right.
- Requires $2n$ expected text character comparisons.

2. Knuth-Morris-Pratt Algorithm:

- Uses a pre-processing to create a partial match table to reduce the number of comparisons.
- Efficient in handling large texts with relatively small patterns.

•Performance:

The performance of the algorithms is compared using text files of 1MB and 2MB sizes. The study specifically compares the Rabin-Karp and the improved Rabin-Karp (IRK) algorithms:

- For a 1MB text file, Rabin-Karp took 100750 microseconds, while IRK took 95500 microseconds.
- For a 2MB text file, Rabin-Karp took 260750 microseconds, while IRK took 175250 microseconds.

Advantages:

Brute Force:

- Simple and easy to implement.
- Works well for small texts and patterns.

Knuth-Morris-Pratt:

- Reduces the number of comparisons needed.
- Pre-processing helps in efficient pattern matching

Disadvantages:

Brute Force:

- Inefficient for large texts due to its linear scanning approach.
- High number of comparisons.

Knuth-Morris-Pratt:

- Pre-processing can be time-consuming.
- Not always the fastest for small patterns or texts.

2.2 "Predicting Gene Expression from Sequence: A Reexamination" [2]

It published in PLOS Computational Biology reexamines the methodologies for predicting gene expression levels from DNA sequences. The study focuses on the biological relevance and prediction accuracy of transcription factor binding sites (TFBS) orientation and position.

Methodology:

The methodology used in this study involves comparing two different procedures for predicting gene expression. Procedure One includes the use of the M198 motif with specific orientation constraints, while Procedure Two uses the RAP1 motif without orientation constraints. The study analyzes the false positive and false negative rates for both procedures to evaluate their performance.

Performance:

The performance of the two procedures was evaluated primarily based on the false positive and false negative rates in predicting gene expression. Procedure One showed a 30% false positive and 18% false negative rate for cluster 1, while Procedure Two had a 38% false positive and 34% false negative rate for the same cluster. This indicates that Procedure One, which includes the M198 motif, is more accurate for cluster 1.

Advantages:**- Procedure One:**

- Lower false positive and false negative rates for cluster 1 compared to Procedure Two.
- The inclusion of the M198 motif appears to improve prediction specificity for certain gene clusters.

Disadvantages:**- Procedure One:**

- The orientation constraint of the M198 motif did not improve global prediction accuracy.
- While it reduces false positive rates for cluster 1, it increases false positive rates in other clusters.
- Indicates limitations in using an automatic "learning machine" for reliable inference under a global classification accuracy criterion.

2.3 "Efficient Pattern Matching Algorithms for Biological Sequences" [3]

Peyman Neamatollahi, Montassir Hadi, and Mahmoud Naghibzadeh addresses the challenge of efficiently searching for patterns in large DNA sequences. The authors introduce three novel pattern matching algorithms designed to enhance performance by leveraging word processing and searching for the least frequent word of the pattern.

Methodology

The paper presents three algorithms, each with a preprocessing and matching phase:

1. First-Last Pattern Matching Algorithm (FLPM):

- Preprocessing Phase: Identifies windows by simultaneously checking the first and last character of the pattern within the text.
- Matching Phase: Verifies these windows for complete pattern matches.

2. Word-Based Pattern Matching Algorithm:

- Utilizes the word processing capability of modern processors to compare 4 or 8 bytes of data simultaneously, improving performance by reducing the number of detected windows and speeding up comparisons.

Performance

The algorithms are evaluated in terms of time cost. The experimental results demonstrate that the proposed algorithms outperform existing ones, particularly in large DNA sequence searches.

Advantages

FLPM:

- Reduces preprocessing time by identifying windows with one pass of the text.
- Efficient for large datasets due to fewer comparisons needed.

Word-Based Algorithm:

- Leverages modern processor capabilities for faster word comparisons.
- Significantly reduces the number of windows to be checked, improving time efficiency

Disadvantages

FLPM:

- While it improves over DCPM, it may still face limitations with extremely large or complex datasets.

Word-Based Algorithm:

- Dependency on processor capabilities means performance gains might vary across different hardware.

2.4 “Digital Design and Computer Architecture” [4]

This chapter covers various aspects of digital design and computer architecture with a focus on combinational and sequential logic circuits. It provides detailed explanations on how to use Verilog to model these circuits, emphasizing the importance of correctly writing Verilog code to simulate accurate hardware behaviour.

Methodology

Combinational Logic

Case Statements: Used to imply combinational logic by describing all possible input combinations. This includes using default statements when necessary.

Priority Cases: Implements logic with priority using the `cases` statement to handle "don't care" conditions (`?`).

Blocking vs. Nonblocking Assignments: Differentiates between non blocking (`<=`) and blocking (`=`) assignments, explaining their respective use cases in synchronous and asynchronous logic.

Signal Assignment Rules: Highlights best practices for signal assignments in Verilog, including using `always` for synchronous sequential logic and `always comb` for complex combinational logic.

Sequential Logic

Flip-Flops and Latches: Describes how to model D flip-flops, including resettable and enable versions, using `always` constructs. Emphasizes the correct use of synchronous (`always@(posedge clk)`) and asynchronous (`always @(posedge clk, pos edge reset)`) resets.

Testbench Creation: Self-checking Testbench: Outlines the steps to create a testbench that applies inputs and checks outputs against expected results.

Test vectors: Uses a file to store test vectors, which are then read and applied in the testbench. This includes generating a clock, reading test vectors into an array, assigning inputs, and comparing outputs.

Performance

The chapter does not directly address performance metrics but implies that following the outlined methodologies ensures correct and efficient digital design. The use of proper Verilog constructs and testbenches helps in verifying the functionality and timing of the designs, leading to optimized performance in real hardware implementations.

Advantages

Clarity and Correctness: Using specialized forms like `always`, `always comb`, and `always latch` helps in writing clear and correct Verilog code that accurately describes the intended hardware.

Error Checking: Tools like `always` check for common mistakes, ensuring the code doesn't inadvertently imply incorrect hardware.

Modular Testing: The methodology for creating testbenches with test vectors ensures thorough testing of the designs, catching errors early in the simulation phase.

Disadvantages

Complexity: The detailed rules and specific constructs required for correct Verilog coding can be complex and may have a steep learning curve for beginners.

Verbose Code: Describing every possible input combination in case statements or managing large test vector files can result in verbose and sometimes cumbersome code.

2.5 "8-bit Smith-Waterman DNA Sequence Alignment Accelerator on ASIC"[5]

Methodology

- Implements 8-bit Smith-Waterman DNA sequence alignment using an ASIC design flow.

- **Two main modules:**

1. **Comparator and Matrix Filling (CMF) Module** – Computes the SW scoring matrix.

2. **Tracing and Scoring (TS) Module** – Finds the highest-scoring alignment path.

- Uses Verilog for hardware description, verified with Xilinx ISE 10.1, synthesized using Design Compiler (DC), and implemented with IC Compiler (ICC).

- Static Timing Analysis (STA) with PrimeTime (PT) ensures efficient timing constraints.

Advantages

High accuracy – Smith-Waterman ensures optimal local alignment.

Parallel processing – Improves execution speed using pipelined architecture.

Low power consumption – ASIC implementation optimizes energy efficiency.

Memory efficiency – Uses divide-and-conquer to reduce memory usage.

Optimized timing – Static timing analysis ensures faster processing.

Disadvantages

Complex design – Requires extensive hardware optimization.

Longer development time – ASIC implementation is slower than FPGA.

Less flexible – ASICs are not reprogrammable like FPGAs.

High cost– ASIC design is expensive and not ideal for small-scale projects.

3. PROBLEM DEFINITION

The objective of this report is to develop and validate a sequence detection system for identifying specific DNA expression patterns within a given sequence of genetic data and to identify the maximum number of character matches between a DNA sequence of **target disease** and a long **input DNA sequence** composed of adenine (A), thymine (T), cytosine (C), and guanine (G).

This system will use a Sequence Detector to process the input sequence, where each character is represented as a 7-bit ASCII value. Instead of searching for an exact match, the system will determine the highest number of character matches between the DNA sequence of target disease within the input DNA sequence obtained from the dataset.

To ensure **versatility** and **adaptability**, the input sequences will be extracted from a dataset stored in a file, allowing the system to operate on different genetic data samples efficiently. This approach enables dynamic testing with various DNA sequences, making the system suitable for broader bioinformatics applications.

The primary goal is to identify the region within the input sequence that has the highest similarity to the target sequence, allowing for the detection of genetic patterns even in the presence of minor variations, insertions, or deletions.

4. PROPOSED WORK

4.1: 7-Bit ASCII Character ATGC Sequence Matching Length Counter for DNA Expression

.This report explains the implementation and functionality of a 7-bit ASCII-based sequence-matching counter for DNA sequence expressions. The design leverages Verilog code for real-time sequence detection and counting matches in DNA data streams encoded as ASCII characters.

4.2: 7-bit ASCII Character ATGC Sequence Matching Length Counter for DNA Expression Using File Input

This report discusses the design and implementation of a system to match DNA sequences using 7-bit ASCII character encoding. The DNA data and target sequences are extracted from external files, allowing flexibility and scalability for handling large datasets. The design, written in Verilog, incorporates sequence matching functionality and includes a testbench for simulation and verification. This report explains the implementation and functionality of a 7-bit ASCII-based sequence-matching counter for DNA sequence expressions. The design leverages Verilog code for real-time sequence detection and counting matches in DNA data streams encoded as ASCII characters.

4.3: 7-bit ASCII Character ATCG Sequence Highest Similarity Detection for DNA Expression Using Smith-Waterman Algorithm

This work proposes a hardware implementation of 7-bit ASCII character ATCG sequence highest similarity detection using the Smith-Waterman algorithm for DNA sequence alignment. The system processes an input DNA sequence from a dataset and compares it against a target DNA sequence of disease to identify the most similar subsequence. Using dynamic programming, a scoring matrix (dp) is updated based on match, mismatch, and gap penalties, ensuring optimal alignment. The highest similarity score (max_score) is dynamically tracked as the sequence is processed. The testbench module loads DNA sequences from files, simulates the system, and logs results in an output CSV file. This hardware-based approach enables real-time sequence alignment, making it efficient for high-speed DNA analysis applications.

4.4: Comparison of 7-bit ASCII for Small Datasets and 7-bit ASCII with Smith-Waterman for Large Datasets

Table 1: Previous Model vs Current Model

Features	Previous Model (7-bit ASCII for Small Datasets)	Current Model (7-bit ASCII with Smith-Waterman for Large Datasets)
Objective	Detects DNA sequence matches using a shift register and counter.	Implement Smith-Waterman DNA alignment on FPGA for better accuracy
Algorithm Used	Simple character-by-character matching using rolling buffers.	Smith-Waterman dynamic programming for optimal sequence alignment.
Processing Method	Sequential matching using shift register and comparison logic	Matrix-based alignment computation with scoring and traceback.
Memory Usage	Memory inefficient for large datasets, processed sequentially	Optimized memory blocks and matrix storage used with Smith-Waterman, enhancing memory efficiency
Matching Capability	Only supports exact matches, unable to handle mutations, insertions, or deletions	Smith-Waterman algorithm enables local sequence alignment, handling mutations, insertions, and deletions
Encoding Flexibility	Limited to 7-bit ASCII, not suitable for complex encodings	Retains 7-bit ASCII, but optimized to handle large datasets efficiently.
Speed & Parallelism	Sequential character processing; no parallel computation.	Supports parallel processing, making execution significantly faster
Accuracy	Lower accuracy, only detects predefined sequences	High accuracy, as Smith-Waterman finds best local alignment
Application Scope	Basic DNA pattern matching in bioinformatics	Advanced DNA sequence alignment for mutation detection and research
Scalability	Struggles with scalability for large datasets and complex sequences	Highly scalable with FPGA and Smith-Waterman, capable of handling large genomic datasets efficiently
Real-World Application	Limited to basic matching, unsuitable for complex bioinformatics tasks	Suitable for complex bioinformatics applications, including mutation detection, genetic research, and disease analysis, using Smith-Waterman

Key Differences & Improvements in the Current Job

More Robust Algorithm: Uses **Smith-Waterman**, allowing for **mutations, insertions, and deletions**, unlike simple character matching.

Parallel Processing Support: The FPGA design enables **faster execution** compared to the sequential shift-register approach.

Higher Accuracy: The **dynamic programming approach** ensures **optimal sequence alignment** rather than detecting exact matches only.

Better Memory Utilization: Instead of **shift registers**, the new system implements **efficient matrix storage** for better scalability.

4.5: Comparative Analysis of Sequence Matching Techniques

Table 2: Comparison of proposed model with different models

Feature	Proposed Model	Reference [1] (<i>Pattern Matching in Genome Analysis</i>)	Reference [2] (<i>Predicting Gene Expression</i>)	Reference [3] (<i>Efficient Pattern Matching</i>)	Reference [4] (<i>Digital Design & Computer Architecture</i>)	Reference [5] (<i>Smith-Waterman on ASIC</i>)
Algorithm Used	7-bit ASCII Matching & Smith-Waterman Algorithm	Brute Force, Knuth-Morris-Pratt, Rabin-Karp	Motif-Based Pattern Recognition	FLPM & Word-Based Pattern Matching	Verilog-based Digital Design Principles	Smith-Waterman Algorithm (ASIC-based)
Flexibility	Smith-Waterman allows mutations, insertions, and deletions	Only exact pattern matching, no mutation handling	Predicts gene expression based on motifs, some flexibility	Optimized for large sequences, but still exact matching	Focuses on Verilog coding practices	Highly flexible, handles mutations well
Accuracy	High (Smith-Waterman ensures optimal alignment)	Lower (relies on exact text matching)	Moderate (False positives &	Moderate (Reduces processing time but	Ensures correct hardware logic but does not	Very High (Dynamic programming-ba

			negatives present)	limited flexibility)	focus on accuracy	sed optimal alignment)
Computational Complexity	Higher for Smith-Waterman, lower for ASCII matching	Low (Brute Force is inefficient, others are optimized for speed)	High (Involves motif recognition & statistical learning)	Moderate (Optimized for efficiency, but limited in handling mutations)	Depends on Verilog coding efficiency	Very High (Uses extensive computational resources but is hardware optimized)
Scalability	Handles 8000-character DNA sequences using file-based input	Efficient for small-scale text matching	Can be applied to large datasets, but limited by false positives	Optimized for larger datasets but lacks mutation handling	Scalability depends on Verilog implementation	Highly scalable (ASIC implementation for high-speed processing)
Processing Speed	Slower due to Verilog simulation, but flexible for testing	Fast for small sequences, inefficient for large ones	Moderate (depends on dataset complexity)	Optimized for faster execution with word-based matching	Depends on Verilog design efficiency	Very Fast (Parallel processing in ASIC hardware)
Hardware Implementation	Currently software-based (Verilog simulation), planned for FPGA	Software-based (text-matching algorithms)	Software-based (bioinformatics prediction model)	Software-based (optimized for CPU performance)	Verilog-based digital logic design	Hardware-based (ASIC optimized for DNA alignment)
Suitability for DNA Analysis	High (Smith-Waterman aligns sequences efficiently)	Limited (Only works well for exact sequence matches)	Moderate (Used for gene expression but not sequence alignment)	Moderate (Improves performance but lacks mutation handling)	Not focused on DNA analysis directly	High (Designed specifically for DNA alignment)
Future Scope	Can be improved with FPGA/GPU acceleration & machine learning integration	Could be improved with probabilistic matching techniques	Could reduce false positive by making hybrid models	Could benefit from parallel processing for real-time analysis	Can be adapted for bioinformatics applications	ASIC is fast but lacks reconfigurability compared to FPGA

5. METHODOLOGY

5.1: 7-Bit ASCII Character ATGC Sequence Matching Length Counter for DNA Expression

Objective

The objective of this methodology is to develop and validate a 7-bit ASCII character ATCG sequence matching length counter for DNA expression. The system should identify the maximum number of character matches between an 18-character target DNA sequence and a 100-character input DNA sequence. This process involves detecting the number of character matches in a dynamic window, which shifts across the input DNA sequence, and counting the matching length relative to the target DNA sequence.

The methodology involves the design and simulation of a SequenceDetector module using Verilog to perform the DNA sequence matching task. The system works with a target DNA sequence, which is represented as a series of 7-bit ASCII values corresponding to the nucleotides adenine (A), thymine (T), cytosine (C), and guanine (G). The input sequence is processed bit-by-bit, and the system tracks the number of consecutive matching characters between the input and the target sequence, updating the count based on matches.

Key Components of the Methodology

Target DNA Sequence Representation

The target DNA sequence consists of 18 characters, represented as 7-bit ASCII values corresponding to the nucleotides adenine (A), thymine (T), cytosine (C), and guanine (G). Each nucleotide is encoded into a 7-bit binary value, and this sequence is pre-defined in the system for comparison.

Input DNA Sequence

The input DNA sequence is 100 characters long and is dynamically provided to the system in blocks of 7-bit ASCII characters. This sequence represents genetic data and is extracted from a file to ensure versatility for testing with different datasets. The input sequence is processed character by character as it arrives, and the system performs a comparison against the target sequence.

Sequence Detection Logic

The SequenceDetector module compares the input DNA sequence to the target DNA sequence on a character-by-character basis. It shifts the target sequence across the input sequence to check for character matches.

Shift Register: The input sequence is fed into a shift register, which stores the most recent 18 characters (corresponding to the target sequence length) for comparison.

Matching Logic: At each clock cycle, the detector checks for matches between the characters in the shift register and the target sequence. If the characters match, a counter is incremented. The module will track the length of the longest matching subsequence at any point.

Match Count Update

The match count, representing the number of consecutive characters that match, is updated after each comparison. If a mismatch is detected, the counter resets, and the search continues for new matching subsequences.

Output

The RTL diagram of the simulation is shown in fig. 1,2 and 3.

In fig. 4 and 5, the waveforms show the different match length of the DNA sequence at different characters.

Testbench and Simulation

The Testbench module initializes the clk, rst, and dna_char inputs to simulate the operation of the SequenceDetector. The testbench feeds the input DNA sequence to the detector and observes the behavior of the match counter.

The sequence data is provided as a 7-bit ASCII encoded array representing the DNA characters A, T, C, and G. The sequence is simulated in a loop to test the matching functionality.

The system checks if the match count correctly updates for each position of the sliding window in the input sequence, which simulates the matching process across the entire sequence.

[illegible][illegible]

Fig. 3 (Fig. 1, 2 and 3 shows the RTL diagram of ATGC Sequence Matching Length Counter)

Waveforms

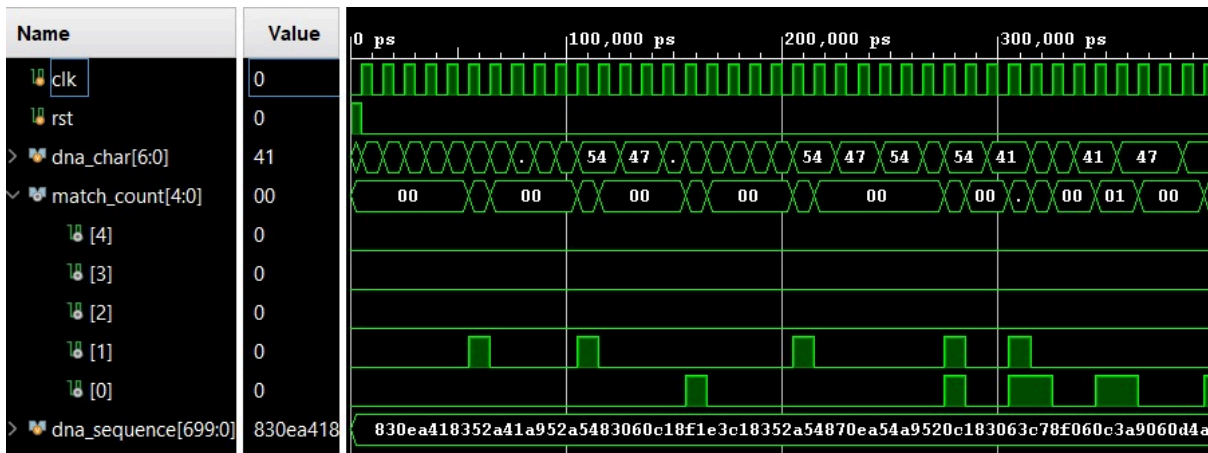


Fig. 4

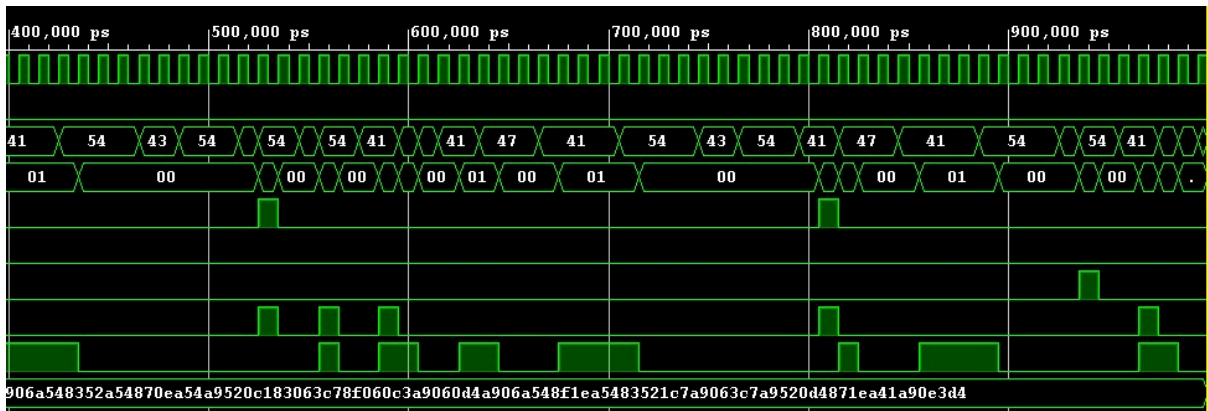


Fig. 5

(Fig . 4 and 5 shows the waveform of ATGC Sequence Matching Length Counter)

5.2: 7-bit ASCII Character ATGC Sequence Matching Length Counter for DNA Expression Using File Input

Objective

The objective of this methodology is to design a 7-bit ASCII character ATCG sequence matching length counter that identifies the maximum number of character matches between an 8000-character input DNA sequence and a 100-character target DNA sequence. The system will extract data from files to process both sequences, enabling a dynamic and flexible approach to sequence matching.

This methodology utilizes a SequenceDetector module written in Verilog to perform DNA sequence matching. The input DNA sequence (8000 characters) and the target DNA sequence (100 characters) are represented in 7-bit ASCII format corresponding to the nucleotides adenine (A), thymine (T), cytosine (C), and guanine (G). The goal is to detect the number of matching characters between the target sequence and a sliding window of 100 characters within the 8000-character input sequence.

Key Components of the Methodology

Target DNA Sequence Representation

The target DNA sequence consists of 100 characters. These characters are encoded in 7-bit ASCII format, where each nucleotide (A, T, C, G) is represented by a specific 7-bit value. The target sequence is pre-stored in the target.txt file, from which it is read and processed.

Input DNA Sequence Representation

The input DNA sequence consists of 8000 characters, which are also represented in 7-bit ASCII format. This sequence is stored in the dna_char.txt file, from which the sequence data is read and processed. The sequence is divided into 7-bit segments and fed into the detector module one character at a time.

Sequence Detection Logic

The SequenceDetector module performs the matching process. It compares the target DNA sequence with the current 100-character subsequence of the input DNA sequence:

Shift Register: The system uses a shift register to store the most recent 100 characters of the input sequence for comparison with the target sequence.

Matching Logic: For each new input DNA character, the sequence detector shifts the sequence window by one character, compares each of the 100 characters against the target sequence, and increments the match counter whenever a character in the sliding window matches the corresponding character in the target sequence.

Match Count Logic

The detector counts the number of consecutive matching characters. If a mismatch is detected, the counter resets. The maximum match count (i.e., the longest matching subsequence) is tracked.

File Handling for Input and Target Sequences

The dna_char.txt file holds the 8000-character input DNA sequence, which is read and loaded into the system.

The target.txt file holds the 100-character target DNA sequence, which is also read into the system.

These files provide the necessary flexibility, allowing for testing with different sequences.

Testbench and Simulation

The Testbench module simulates the operation of the SequenceDetector. It initializes the clock (clk), reset (rst), and input DNA character (dna_char) signals.

It reads the sequences from files, processes each character from the input sequence, and tests the sequence detection logic.

A CSV output file (output.csv) is generated to record the simulation results, including time, reset state, clock state, input character, and match count.

Output

The RTL diagram of the simulation is shown in fig. 6,7 and 8.

In fig. 9 and 10, the waveforms show the different match length of the DNA sequence at different characters which have been taken from file.

The results of the 'output.csv' file are shown in table 3 and 4, where it is shown the time and character where the whole target sequence matched.

Timing Control

The clock (clk) operates with a period of 10 time units, and the system updates every 10 time units.

After reading the input sequence, the system continues to check for matches until the entire input sequence has been processed.

RTL Diagram

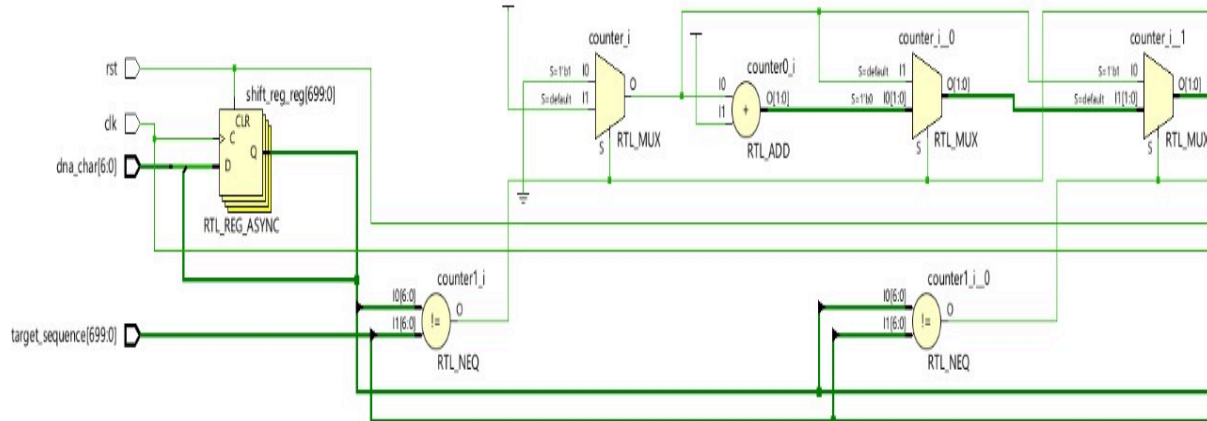


Fig. 6

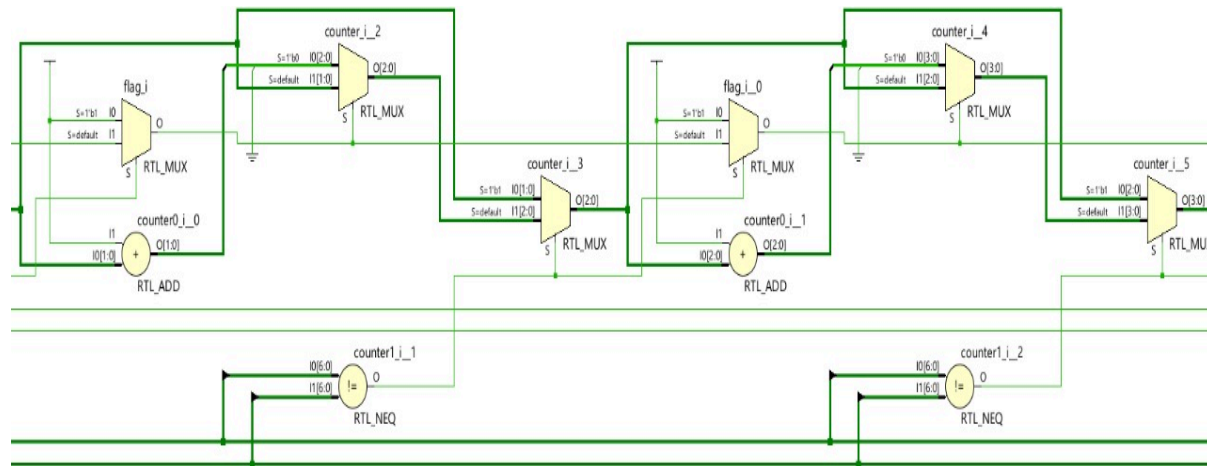


Fig. 7

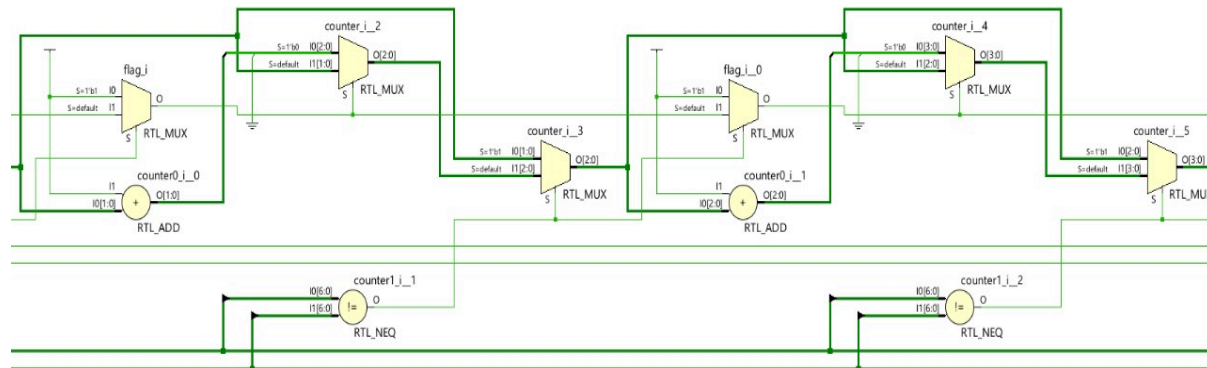


Fig. 8

(Fig . 6,7 and 8 shows the RTL diagram of ATGC Sequence Matching Length Counter)

WaveForm

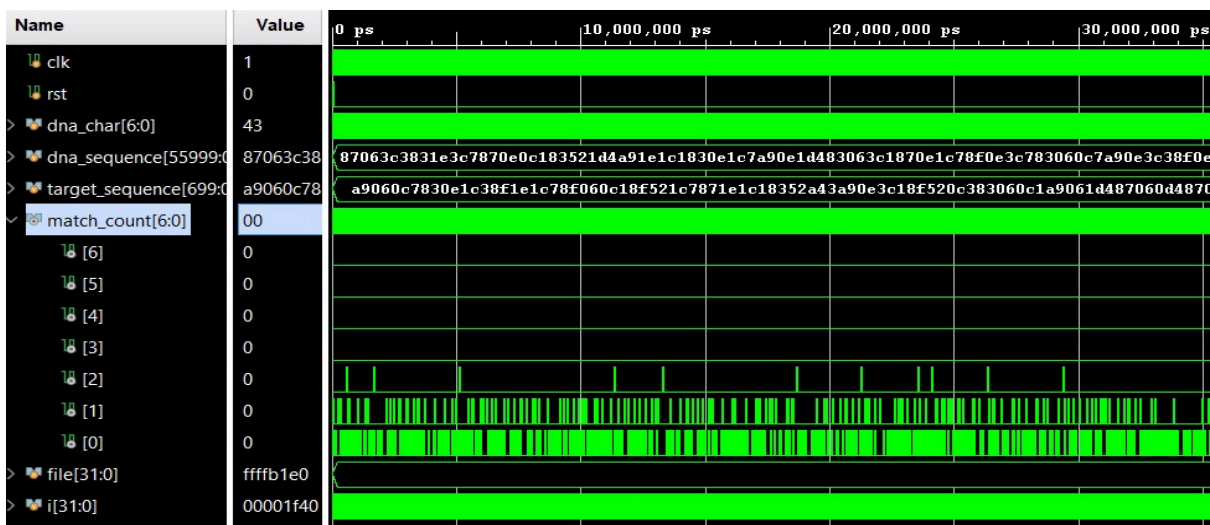


Fig. 9

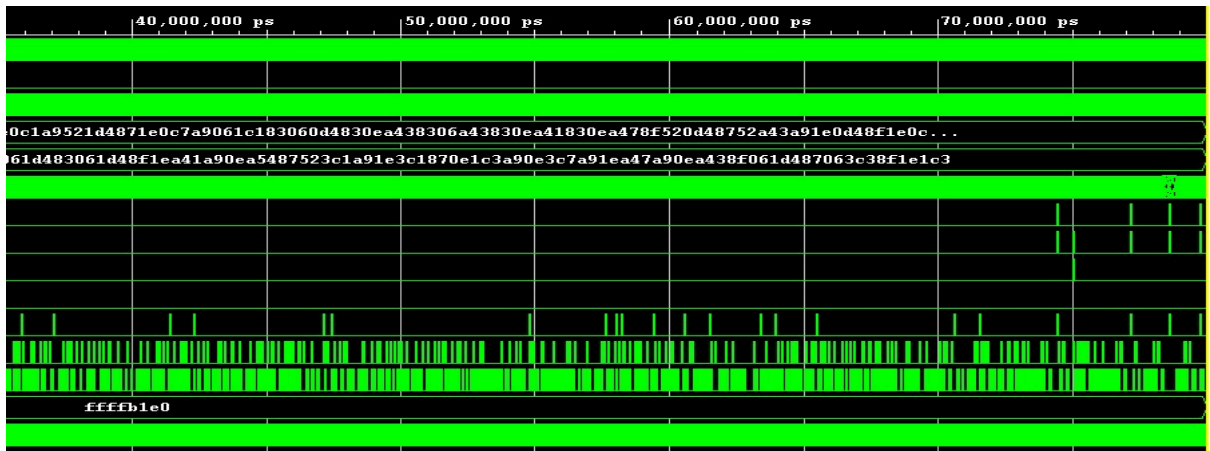


Fig. 10

(Fig . 9 and 10 shows the waveform of ATGC Sequence Matching Length Counter)

Sample of Result

Table 3: Results of output.csv file

	A	B	C	D	E
1	time	rst	clk	dna_char	match_count
2	0	1	0	0	x
3	5000	1	1	0	0
4	10000	0	0	67	0
5	15000	0	1	67	0
6	20000	0	0	71	0
7	25000	0	1	71	0
8	30000	0	0	65	0
9	35000	0	1	65	0

Table 4

	A	B	C	D	E
14882	74400000	0	0	65	0
14883	74405000	0	1	65	0
14884	74410000	0	0	65	100
14885	74415000	0	1	65	100
14886	74420000	0	0	65	0

5.3: 7-bit ASCII Character ATCG Sequence Highest Similarity Detection for DNA Expression Using Smith-Waterman Algorithm

Objective

The goal of this is to implement a 7-bit ASCII character sequence with highest similarity detection using the Smith-Waterman algorithm. The algorithm will find the most similar subsequence between an input DNA sequence from a dataset and a DNA sequence of target disease. This solution will also include dynamic programming for sequence alignment.

Key Components of the Methodology

5.3.1. Target DNA Sequence extraction from dataset

The target DNA sequence of disease, encoded as 7-bit ASCII values. Each nucleotide (A, T, C, G) is represented by a unique 7-bit value. This sequence is stored in the covid_dna.txt file.

5.3.2. Input DNA Sequence extraction from dataset

The input DNA sequence in 7-bit ASCII format, stored in the chimpanzee_dna.txt file. The sequence is fed to the sequence detector one character at a time.

5.3.3. Smith-Waterman Algorithm Implementation

The **Smith-Waterman algorithm** is a dynamic programming algorithm used for **local sequence alignment**, meaning it identifies regions of similarity between two sequences (e.g., DNA, RNA, or proteins) while allowing for gaps and mismatches.

Key Features:

- 1. Local Alignment:** Unlike global alignment (e.g., Needleman-Wunsch), Smith-Waterman finds the best matching subsequence, ignoring non-matching regions.
- 2. Gap Penalties:** It accounts for insertions and deletions with penalties.
- 3. Scoring Matrix:** Uses a substitution matrix (like BLOSUM for proteins) to assign match/mismatch scores.
- 4. Traceback:** Finds the optimal alignment by backtracking from the highest-scoring cell.

Algorithm Steps:

1. Initialization:

- Create scoring matrix **H** of size $((m+1) * (n+1))$ for sequences **A** (length m) and **B** (length n).
- Initialize the first row and first column to **zero**, allowing local alignment.

2. Fill the Scoring Matrix:

- Use the recurrence relation:

$H_{i-1,j-1} + s(x_i, y_j) \nearrow$	$H_{i-1,j} - W_1 \downarrow$
$H_{i,j-1} - W_1 \rightarrow$	$H_{i,j}$

$$H(i, j) = \max \begin{cases} 0, \\ H(i-1, j-1) + S(x_i, y_j) & \text{(Diagonal: Match/Mismatch Score)} \\ H(i-1, j) - d & \text{(Up: Deletion Penalty)} \\ H(i, j-1) - d & \text{(Left: Insertion Penalty)} \end{cases}$$

where

- **H(i,j)** = Score at position (i, j).
- **S(x_i, y_j)** is the substitution score.
- **d** is the gap penalty.
- If a cell has a **negative value**, it is set to **zero** to indicate the start of a new alignment.

3. Traceback:

- Start from the **highest-scoring cell**.
- Move diagonally for a match/mismatch or horizontally/vertically for gaps.
- Stop when a **zero** is encountered.

4. Output the Best Alignment:

- Extract the highest-scoring local subsequence.

Example Calculation:

Given:

	-	A	C
-	0	0	0
A	0	2	0
G	0	0	0
C	0	0	4

- Sequence **A** = "AGC"
- Sequence **B** = "AC"
- Match = +2, Mismatch = -1, Gap = -2.
- The **highest score** = 4, found at (3,2), which corresponds to C aligned with C.
- Traceback results in "AC" aligned with "AC", ignoring "G".

Advantages:

- Identifies local similarities even if sequences have large non-matching regions.
- Handles substitutions, insertions, and deletions effectively.
- Widely used in bioinformatics for gene comparison, protein analysis, and evolutionary studies.

Disadvantages:

- Computationally expensive (Time Complexity: $O(mn)$).
- Not suitable for long sequences due to high memory usage.

5.3.4. Dynamic Programming Table

MATCH_SCORE=1

MISMATCH_PENALTY=-1

GAP_PENALTY=-2

The dp and prev_dp registers hold the values of the dynamic programming table at the current and previous steps, respectively. This allows for efficient computation of sequence alignments.

5.3.5. Testbench for Simulation

A testbench is provided to simulate the operation of the SequenceDetector module. It loads DNA sequences from files, processes each character, evaluates the highest similarity score.

5.3.6. File Handling for Input and Target Sequences

covid_dna.txt: Contains the DNA sequence of target disease. [D1]

chimpanzee_dna.txt: Contains the input DNA sequence. [D2]

human_dna.txt: Contains the input DNA sequence. [D3]

5.3.7. Output

The RTL diagram of the simulation is shown in fig. 11,12 and 13.

In fig. 14 and 15, the waveforms show that the maximum match between the input DNA sequence of Chimpanzee and target sequence of Covid is 31.

The results of the 'output.csv' file are shown in table 5 and 6, where it is shown the time and character where the maximum match of 31 characters occurred.

5.3.8. Timing Control

The clock (clk) operates with a period of 10 time units, and the system updates every 10 time units.

After reading the input sequence, the system checks for alignments and calculates the highest similarity score.

RTL Diagram

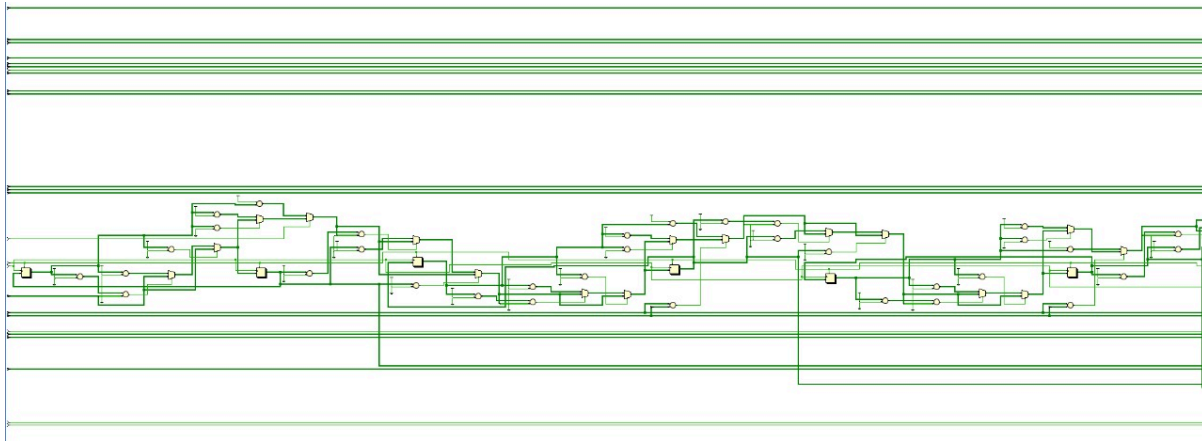


Fig. 11

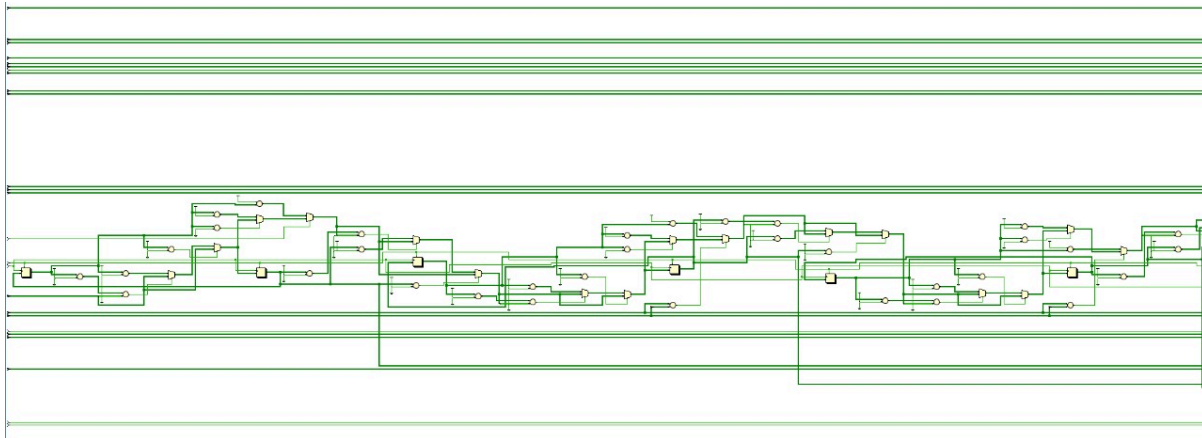


Fig. 12

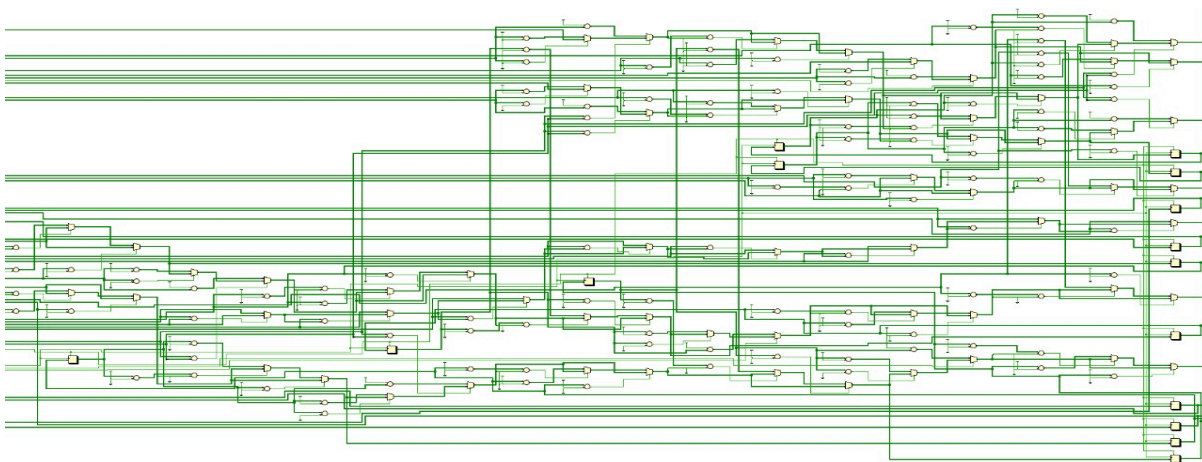


Fig. 13

(Fig . 11,12 and 13 shows the RTL diagram of ATGC Sequence Highest Similarity Detection for DNA Expression)

WaveForm

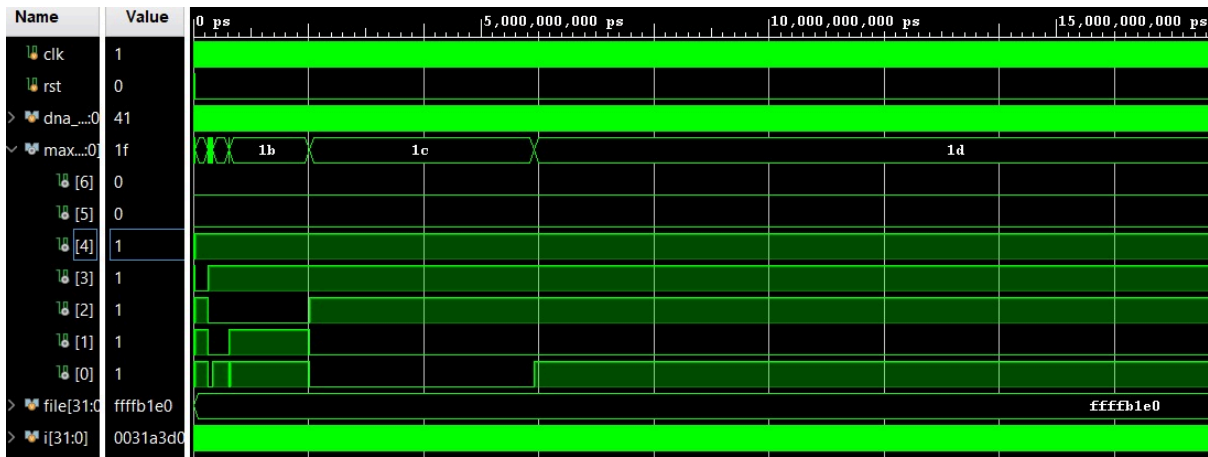


Fig. 14

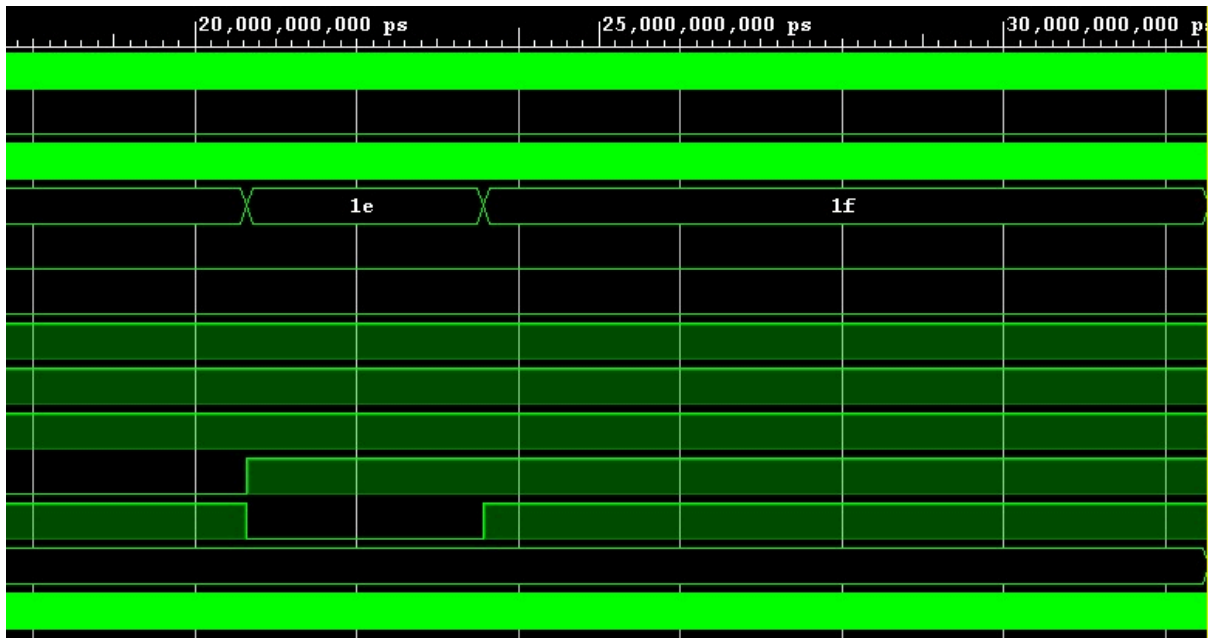


Fig. 15

(Fig . 14 and 15 shows the waveform of ATGC Sequence Highest Similarity Detection for DNA Expression of first DNA sequence)

Sample of Output

Table 5: Results of output.csv file of first simulation

24175210000	0	0	71	30
24175215000	0	1	71	30
24175220000	0	0	67	30
24175225000	0	1	67	30
24175230000	0	0	65	30
24175235000	0	1	65	30
24175240000	0	0	84	31
24175245000	0	1	84	31
24175250000	0	0	71	31
24175255000	0	1	71	31
24175260000	0	0	65	31
24175265000	0	1	65	31

Table 6: Results of output.csv file of 2nd simulation

24175210000	0	0	71	30
24175215000	0	1	71	30
24175220000	0	0	67	30
24175225000	0	1	67	30
24175230000	0	0	65	30
24175235000	0	1	65	30
24175240000	0	0	84	31
24175245000	0	1	84	31
24175250000	0	0	71	31
24175255000	0	1	71	31
24175260000	0	0	65	31
24175265000	0	1	65	31

Table 7: Results of output.csv file of 3rd simulation

25139420000	0	0	71	1210
25139425000	0	1	71	1210
25139430000	0	0	84	1210
25139435000	0	1	84	1210
25139440000	0	0	67	1211
25139445000	0	1	67	1211
25139450000	0	0	67	1211
25139455000	0	1	67	1211
25139460000	0	0	65	1211

Table 8: Results of output.csv file of 4th simulation

24971120000	0	0	71	986
24971125000	0	1	71	986
24971130000	0	0	84	986
24971135000	0	1	84	986
24971140000	0	0	71	987
24971145000	0	1	71	987
24971150000	0	0	84	987
24971155000	0	1	84	987
24971160000	0	0	65	987

Table 9: Overall Results of 4 simulations

Sl. No.	Animal DNA Sequence Used	Covid Genome Used	Maximum Match Found	Simulation Time-Stamp	DNA Sequence Index where the result is found
1	Chimpanzee	JF292909 SARS coronavirus MA15 isolate d2ym4 complete genome	31	23568000000	2,356,799
2	Chimpanzee	JQ316196 SARS coronavirus HKU-39849 isolate UOB complete genome	31	24175240000	2,417,523
3	Human	MK129253 Homo sapiens South Korea 2018/10/20	1211	25139440000	2,513,943
4	Human	MH822886 Homo sapiens United Kingdom 2018/08/22	987	24971140000	2,497,113

6. CONCLUSION

The field of DNA sequence matching has evolved significantly, driven by advances in computational techniques and the growing availability of genomic data. These developments have enabled more accurate and efficient disease detection, with profound implications for genetic testing, cancer research, infectious disease diagnostics, and pharmacogenomics. Continued innovation in algorithms and methodologies will further enhance our ability to understand and leverage genetic information for improved health outcomes.

7. FUTURE SCOPE

Potential areas for enhancing the sequence detection system will be identified, such as optimizing the algorithm for larger datasets, improving processing speed, and increasing detection accuracy. Recommendations for future research and development based on the findings and performance evaluation will be provided.

The successful completion of this project will demonstrate the feasibility of using a Sequence Detector for DNA sequence matching encoded in 7-bit ASCII format. This will provide a foundation for more advanced genetic analysis tools and techniques. This report will present the design, implementation, and evaluation of the system, along with a discussion on its potential impact and applications in bioinformatics.

We are planning to use FPGA for DNA sequence matching. As the dataset for sequence matching is very vast so FPGA appears to be the best tool. Its ability to provide high speed, accurate, parallel and energy efficient processing makes it an ideal choice for the next generation of genomic technologies.

DATASETS

[1]. DNA sequence of target disease

<https://www.kaggle.com/datasets/pushkar007/covid-deeppredictor>

[2] Input DNA sequence of chimpanzee genome

<https://www.kaggle.com/datasets/nageshsingh/dna-sequence-dataset>

[3] Input DNA sequence of human genome

<https://www.kaggle.com/datasets/nageshsingh/dna-sequence-dataset/data?select=human.txt>

8. REFERENCES

- [1] Gope, Ashish Prosad, and Rabi Narayan Behera. "A Novel Pattern Matching Algorithm in Genome Sequence Analysis." *IJCSIT) International Journal of Computer Science and Information Technologies* 5.4 (2014): 5450-5460.
- [2] Neamatollahi, Peyman, Montassir Hadi, and Mahmoud Naghibzadeh. "Simple and efficient pattern matching algorithms for biological sequences." *IEEE Access* 8 (2020): 23838-23846.
- [3] Yuan, Yuan, et al. "Predicting gene expression from sequence: a reexamination." *PLoS computational biology* 3.11 (2007): e243.
- [4] Harris, David Money, and Sarah L. Harris. "Hardware description languages." *Digital Design and Computer Architecture* (2022): 170-235.
- [5] Halim, A. K., et al.. Design and Analysis of 8-bit Smith Waterman based DNA Sequence Alignment Accelerator's Core on ASIC Design Flow (2010). IEEE. DOI: [10.1109/EMS.2010.31]
- [6] Oliveira, F. F. de, Dias, L. A., & Fernandes, M. A. C. . Parallel Implementation of Smith-Waterman Algorithm on FPGA.(2021) bioRxiv. DOI: 10.1101/2021.07.27.454006
- [7] Mahmod, N. B. (Year Unknown). Design and Analysis of High Performance Matrix Filling for DNA Sequence Alignment Accelerator Using ASIC Design Flow. Faculty of Electrical Engineering, Universiti Teknologi MARA.
- [8] Yusoff Cheah, R. A., Halim, A. K., Al Junid, S. A. M., & Khairudin, N. . Design and Analysis of Low Powered DNA Sequence Alignment Accelerator Using ASIC Design Flow.(2010)